MCP Server Boilerplate

Production-ready template · Dual Auth (OAuth 2.0 + API Key) · FastMCP + FastAPI

- Python 3.10+
- FastMCP
- FastAPI
- OAuth 2.1
- RFC 7591 DCR
- Azure Entra

Two Auth Schemes

Bearer JWT (OAuth 2.0) for interactive users
API Key for server-to-server calls

MCP Compliant

RFC 9728 · RFC 8414 · RFC 7591
Streamable HTTP transport

(https://modelcontextprotocol.io/docs/tutorials/security/authorization)

Drop-in Template

Add tools in server.py
Auth & discovery wired out of the box

Architecture Overview

HTTP Request

AuthenticationMiddleware
Bearer JWT | x-api-key

MCPPathMiddleware
/mcp → /mcp/ rewrite

FastAPI Router
/.well-known · /register /authorize /token · /mcp

FastMCP
Your MCP tools execute here

Key Files

`auth.py`
Auth gate — JWT or API key

`jwt.py`
Entra token validation

`oauth_proxy.py`
DCR + authorize + token proxy

`well_known.py`
RFC 8414 / RFC 9728 endpoints

`server.py`
FastMCP + your MCP tools

`auth_context.py`
AuthContext via contextvars

⚡ Raw ASGI middleware — not BaseHTTPMiddleware

Two Authentication Schemes

① API Key

Simple header-based auth. No Azure setup required.

```
# Request
x-api-key: your-secret-key

# .env
API_KEYS=key-one,key-two
```

Use cases

- Server-to-server integration
- Local development (no Azure)
- CI/CD pipelines and scripts

`auth_type = "api_key"` · `user_oid = None`

② OAuth 2.0 Bearer Token

Azure Entra JWT — full user identity in every request.

```
Authorization: Bearer eyJ0eXAiOiJKV1Q...

"oid": "b6167420-..."  # user ID
"scp": "access_as_user"
```

Use cases

- Claude.ai / Claude Desktop / ChatGPT users
- Copilot Studio / Azure Foundry connectors
- Any OAuth 2.1 client

`auth_type = "bearer"` · `user_oid / user_name / user_upn`

Auth Middleware — Decision Flow

Incoming HTTP Request

Exempt path?
/.well-known /health /register /authorize /token

Bearer token present?
Validate JWT via Entra JWKS

x-api-key present?
Validate against API_KEYS env var

No credentials → 401
WWW-Authenticate: Bearer resource_metadata=...

AuthContext (contextvars)

```
@dataclass
class AuthContext:
    auth_type: str     # "bearer"|"api_key"
    user_oid:  str | None
    user_name: str | None
```

✓ Set AuthContext (bearer)

✓ Set AuthContext (api_key)

OAuth 2.0 — Auto-Discovery Chain

Clients discover all OAuth endpoints automatically — no manual configuration needed.

1 **Client hits /mcp without credentials**

```
POST /mcp  →  401 WWW-Authenticate: Bearer
        resource_metadata=.../.well-known/oauth-protected-resource
```

2 **Fetch RFC 9728 Protected Resource Metadata**

```
GET /.well-known/oauth-protected-resource
← { "resource": "https://host/mcp", "authorization_servers": ["https://host"] }
```

3 **Fetch RFC 8414 Authorization Server Metadata**

```
GET /.well-known/oauth-authorization-server
← { "issuer": "https://host", "authorization_endpoint": "...", "registration_endpoint": "..." }
```

4 **Proceed with standard OAuth 2.1 flow**

```
DCR → /register  ·  Authorization → /authorize  ·  Token exchange → /token
```

OAuth Proxy — Azure Entra Bridge

Entra doesn't support RFC 7591 DCR. The proxy bridges this gap — clients see a generic OAuth 2.1 server.

① **POST /register (RFC 7591 DCR)**

Client sends {redirect_uris, client_name} → server returns {client_id: "dynamic-xxxx"}

② **GET /authorize — swap & inject**

client_id → AZURE_CLIENT_ID · inject scope: api://xxx/access_as_user · drop resource param → 302 Entra

③ **User logs in at Entra**

Authorization code returned to redirect_uri

④ **POST /token — swap credentials**

client_id/secret → AZURE_CLIENT_ID/SECRET · drop resource · forward to Entra /oauth2/v2.0/token

⑤ **Entra returns JWT — validated by middleware**

access_token.aud = "api://AZURE_CLIENT_ID" ✓

Clients see

› Generic OAuth 2.1 server

› Standard DCR endpoint

› No Azure-specific scopes

› No Entra URLs

Entra sees

› Real AZURE_CLIENT_ID

› Real AZURE_CLIENT_SECRET

› Correct api:// scope

› No resource parameter

JWT Token Validation (Resource Server Role)

```python
async def authenticate(token: str):

    # 1. Extract kid from token header
    kid = jwt.get_unverified_header(token)["kid"]

    # 2. Fetch Entra JWKS (cached 1 hour)
    public_key = rsa_pem_from_jwk(fetch_jwks(kid))

    # 3. Verify signature + audience + expiry
    payload = jwt.decode(
        token, public_key,
        algorithms=["RS256"],
        audience=valid_audiences,
    )

    # 4. Manual issuer check (v1 + v2 tokens)
    if payload["iss"] not in valid_issuers: raise ...

    # 5. Return user identity
    return payload["oid"], payload.get("name"), payload.get("upn")
```

Accepted Audiences	
`api://{CLIENT_ID}`	Standard App ID URI
`{CLIENT_ID} GUID`	Raw GUID / id tokens
`{BASE_URL}`	RFC 8707 resource URI

Accepted Issuers	
`login.microsoftonline.com`	User tokens (v2.0)
`sts.windows.net`	App tokens (v1)

Error Codes → HTTP	
`ExpiredSigError`	401
`InvalidAudienceError`	403
`InvalidIssuerError`	403
`InvalidTokenError`	401

Stream Concurrency Control

Each MCP tool call holds a streaming HTTP connection open — limits prevent resource exhaustion.

Incoming Request

Semaphore.acquire(timeout=QUEUE_WAIT_TIMEOUT)

Timeout → 429 Too Many Requests
Retry-After header included

asyncio.create_task(app_runner)

idle_watchdog
Cancel if no bytes sent > IDLE_STREAM_TIMEOUT

hard_watchdog
Cancel after HARD_STREAM_TIMEOUT regardless

CancelledError → 503 · finally: Semaphore.release()

Configuration (.env)

MAX_CONCURRENT_STREAMS	20 — semaphore size
QUEUE_WAIT_TIMEOUT	5 s — wait before 429
HARD_STREAM_TIMEOUT	300 s — max request life
IDLE_STREAM_TIMEOUT	60 s — max silence

GET /health → Live Metrics

```
{
  "mcp": {
    "active_streams":3,
    "max_streams": 20
  }
}
```

Getting Started

① Configure .env

```
BASE_URL=https://your-host.com
API_KEYS=your-secret-key

# Azure Entra (OAuth 2.0)
AZURE_TENANT_ID=<tenant-id>
AZURE_CLIENT_ID=<app-reg-id>
AZURE_CLIENT_SECRET=<secret>
```

② Add Tools in src/server.py

```
@mcp.tool()
async def my_tool(param: str) -> str:
    """Description shown to LLM."""
    auth = get_auth()
    return "result"
```

③ Azure App Registration

```
Redirect URI (Web):
  https://claude.ai/api/mcp/auth_callback
Redirect URI (Desktop):
  http://localhost
Expose API scope: access_as_user
```

④ Run

```
# Local dev
uv sync && python dev.py

# Docker
docker-compose up -d

# Health check
curl http://localhost:8080/health
```